Chapitre 5 :
Gestion des modules
&
packages

Modules

Les modules font référence à un fichier contenant des instructions, des fonctions, des classes ou des variables Python.

Avantages des Modules :

- **Réutilisation de code**
- **Organisation et modularité**
- **Facilitation de la collaboration.**

Types des Modules :

- **Modules intégrés** : fournis par Python (ex. : math, os, random).
- **Modules tiers** : disponibles via pip (ex. : numpy, requests).
- **Modules personnalisés** : créés par l'utilisateur (ex. : monmodule.py).

Importer des Modules

1. **import module :** importe tout le contenu du module sous forme d'un objet.

   Exemple :

   ```
   import math
   print(math.sqrt(16)) # Toutes les fonctions et variables du module doivent être appelées avec le préfixe du nom du module.
   ```

2. **from module import function : importe une fonction spécifique depuis le module**

   Exemple :

   ```
   from math import sqrt
   print(sqrt(16)) #Aucune nécessité d'utiliser le préfixe du module pour appeler les fonctions ou variables.
   ```

3. **from module import * : importe toutes les fonctions. À Eviter!**

   Exemple :

   ```
   from math import * #Peut entraîner des conflits si plusieurs modules contiennent des fonctions ou variables avec le même nom.
   print(sqrt(16))
   ```

Utilisation d'alias pour les Modules

L'utilisation d'alias avec **as** en Python permet de renommer un module ou une fonction lors de son importation.

Cela est particulièrement utile pour :

1. **Simplifier les noms de modules/fonctions longs ou compliqués.**
2. **Rendre le code plus lisible ou plus concis.**
3. **Éviter les conflits de noms dans le cas où un autre module ou variable porte le même nom.**

```
# Importation du module math avec un alias
import math as m

# Utilisation du module via son alias 'm'
print(m.sqrt(16))  # 4.0
print(m.pi)        # 3.14159...
```

Les modules couramment utilisés en Python

1- math : Fournit des fonctions mathématiques avancées comme sqrt, sin, cos, log.

2- random : Génère des nombres pseudo-aléatoires, utile pour les simulations ou le mélange des données.

3- datetime : Manipule les dates et heures pour l'analyse temporelle.

4- json : Charge et enregistre des fichiers JSON.

5- numpy : Bibliothèque pour le calcul numérique avec des tableaux multidimensionnels

6- pandas, matplotlib etc…

Introduction à pip

Le gestionnaire de paquets pip est un outil indispensable pour les développeurs Python.

Il permet d'installer, de mettre à jour et de gérer automatiquement des bibliothèques et des modules tiers qui ne sont pas inclus dans la bibliothèque standard de Python.

- La commande de base pour installer un module est :

```
pip install nom_du_module
pip install numpy
```

- La commande pour mettre à jour un module déjà installé :

```
pip install --upgrade nom_du_module
```

- La commander pour désinstaller un module :

```
pip uninstall nom_du_module
```

- La commande pour lister les modules installés:

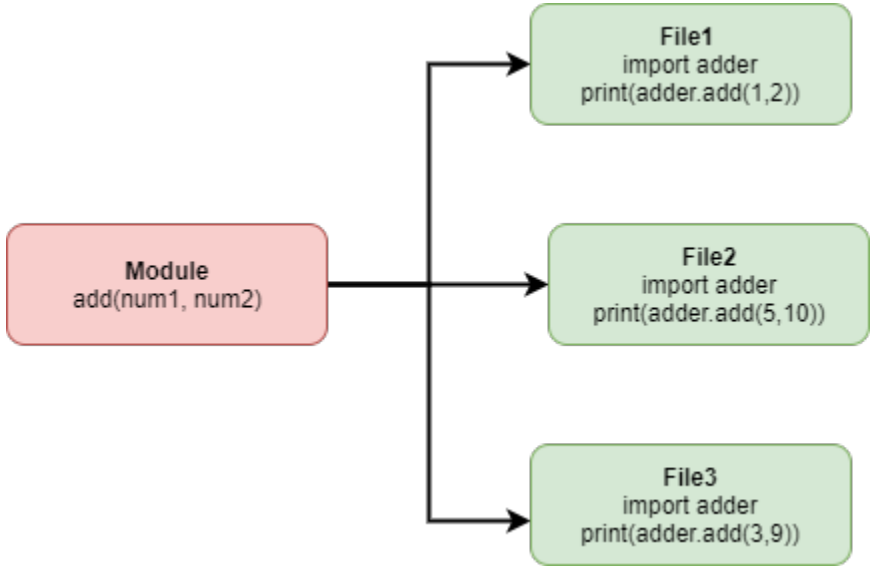
```
pip list
```

Créer et utiliser des Modules personnalisés

Un fichier contenant du code Python, par exemple example.py, est appelé un module, et son nom de module serait example.

Pour importer un module sous un nouveau nom, on utilise la syntaxe suivante :

import mymodule as new_module




```python
# Mudule adder.py

def add(a, b):
    result = a + b
    return result
```

```python
#File1

import adder

print(adder.add(4,5.5))
```

```
9.5
```

Package Python

Un package Python est un répertoire contenant plusieurs modules Python (fichiers .py) et, éventuellement, d'autres sous-packages (sous-répertoires).

Les packages permettent de structurer un projet Python de manière organisée, surtout lorsque le projet devient complexe.

```
project/
├── module1
│   ├── __init__.py
│   └── module1.py
└── module2
    ├── __init__.py
    └── module3
        ├── __init__.py
        └── module3.py
```

Avant Python 3.3 : Un package doit contenir un fichier nommé __init__.py pour que Python le considère comme un package.

Depuis Python 3.3 : Ce fichier est facultatif, mais il reste très utile.

Le fichier __init__.py peut être laissé vide, mais on y place généralement les variables globales ou du code d'importation.

Utilisation des imports dans les Packages

Nous pouvons importer des modules depuis des packages en utilisant l'opérateur point (.)

Exemple :

```
import pkg.sub_pkg1.mod2.py
```